A Hybrid Quantum–Classical Optimization Framework for Well Placement in Oil Reservoirs Using QUBO Modeling and QAOA

Tareq Mageed[1,*]

[1]Ministry of Oil, Iraq
Dhi Qar Oil Company
Quality Management and Institutional Development Division

*Corresponding author: tariq.mageed@gmail.com

Abstract

Optimal well placement is a central decision problem in field development planning because it directly affects production performance and project economics. The combinatorial nature of the problem makes exhaustive search infeasible for large candidate sets. This study presents a hybrid quantum–classical optimization framework for well placement using Quadratic Unconstrained Binary Optimization (QUBO) modeling and the Quantum Approximate Optimization Algorithm (QAOA). The framework reformulates well selection as a binary optimization problem with production rewards and distance-dependent interference penalties. A controlled four-well validation benchmark is used, allowing exact comparison against brute-force enumeration and simulated annealing.

The implementation demonstrates that the QAOA-based solution recovers the global optimum exactly, achieving an approximation ratio of 1.0. The results confirm the mathematical correctness of the QUBO formulation and the algorithmic validity of the QAOA implementation on a fully verifiable benchmark. Supporting visualizations are generated to compare energy, production, interference, and runtime across methods.

1 Introduction

Well placement optimization is a combinatorial problem where the objective is to select a subset of candidate wells that maximizes production while minimizing interference. For N candidate wells, the search space consists of 2^N configurations. Classical metaheuristics

such as simulated annealing are widely used, but quantum-inspired methods offer an alternative approach that can be systematically benchmarked.

The Quantum Approximate Optimization Algorithm (QAOA) is a variational algorithm designed for problems expressible in Ising or QUBO form [1]. In this work, the well placement problem is cast as a QUBO model, and a depth-$p = 2$ QAOA circuit is executed on a classical simulator. The results are compared against brute-force enumeration and simulated annealing. The validation benchmark consists of $N = 4$ wells, which allows complete enumeration of $2^4 = 16$ configurations and provides a rigorous correctness check.

2 Problem Formulation

Let $x_i \in \{0, 1\}$ be the binary decision variable for well i, where $x_i = 1$ indicates selection. The objective energy is defined as:

$$E(x) = x^T Q x \tag{1}$$

where $Q \in \mathbb{R}^{N \times N}$ is the QUBO matrix.

3 QUBO Construction

The diagonal terms of Q encode production rewards:

$$Q_{ii} = -P_i \tag{2}$$

where P_i is the production potential of well i. In the validation benchmark, the production potentials are:

$$P = [0.9,\ 0.3,\ 0.8,\ 0.2] \tag{3}$$

The off-diagonal terms represent spatial interference between wells i and j:

$$Q_{ij} = 0.2 \exp\left(-\frac{d_{ij}}{\lambda}\right), \quad i \neq j \tag{4}$$

where d_{ij} is the Euclidean distance between well coordinates, and $\lambda = 400.0$ is the decay length. The well coordinates used in the benchmark are:

$$\begin{aligned}
&\text{Well 0:} \quad (0.0, 0.0) \\
&\text{Well 1:} \quad (500.0, 0.0) \\
&\text{Well 2:} \quad (0.0, 500.0) \\
&\text{Well 3:} \quad (500.0, 500.0)
\end{aligned}$$

This formulation creates a natural trade-off: selecting high-production wells is favored,

but closely spaced wells incur an interference penalty.

4 Optimization Methods

4.1 Brute-Force Exact Enumeration

Because $N = 4$, all $2^4 = 16$ binary vectors can be evaluated exactly. The global minimum energy E_{best} and maximum energy E_{worst} are recorded to compute approximation ratios.

4.2 Simulated Annealing

The `dual_annealing` routine from SciPy [2] is applied to the continuous relaxation $x_i \in [0, 1]$, with rounding to $\{0, 1\}$ for energy evaluation. The algorithm uses a maximum of 500 iterations and a fixed random seed for reproducibility.

4.3 QAOA Implementation

The QUBO matrix is first mapped to an Ising Hamiltonian with local fields h_i and couplings J_{ij}. The standard transformation from QUBO to Ising is given by:

$$h_i = \frac{1}{2}\left(Q_{ii} + \sum_{j \neq i} Q_{ij}\right) \tag{5}$$

$$J_{ij} = \frac{1}{4} Q_{ij} \tag{6}$$

This mapping is implemented in the code via the `qubo_to_ising` function.

A depth-$p = 2$ QAOA circuit is constructed using Qiskit [3]. The circuit alternates cost unitaries (implemented via R_z and CX gates) and mixing unitaries (R_x gates). Specifically, for each layer ℓ, the cost unitary applies:

$$U_C(\gamma_\ell) = \exp\left(-i\gamma_\ell \left(\sum_i h_i Z_i + \sum_{i<j} J_{ij} Z_i Z_j\right)\right) \tag{7}$$

and the mixing unitary applies:

$$U_B(\beta_\ell) = \exp\left(-i\beta_\ell \sum_i X_i\right) \tag{8}$$

where Z_i and X_i are Pauli operators on qubit i.

The variational parameters $\gamma_1, \gamma_2, \beta_1, \beta_2$ are optimized classically using the COBYLA algorithm [4] with a budget of 40 iterations. Each circuit evaluation uses 1024 shots. The classical optimizer COBYLA was chosen for its derivative-free nature, which is well-suited to noisy objective landscapes in variational quantum algorithms.

After optimization, a final sampling run with 4096 shots is performed. The best solution is extracted as the bitstring with the *lowest observed energy*, not merely the most frequent bitstring. The expected energy reported during optimization is the average energy over the measurement distribution.

5 Validation Benchmark Results

5.1 Exact and Classical Baselines

Brute-force enumeration yields:

$$\begin{aligned} x^* &= [1, 1, 1, 0] \quad \text{(wells 0, 1, 2 selected)} \\ E_{\text{best}} &= -1.702511 \\ E_{\text{worst}} &= 0.000000 \quad \text{(all wells off)} \end{aligned}$$

Simulated annealing recovers the identical configuration:

$$\begin{aligned} x_{\text{SA}} &= [1, 1, 1, 0] \\ E_{\text{SA}} &= -1.702511 \end{aligned}$$

5.2 QAOA Performance

The optimized QAOA circuit produces a distribution over bitstrings. The lowest-energy sampled bitstring is:

$$\begin{aligned} \text{Bitstring:} \quad & 0111 \text{ (reading from Qiskit little-endian ordering)} \\ x_{\text{QAOA}} &= [1, 1, 1, 0] \\ E_{\text{QAOA}} &= -1.702511 \end{aligned}$$

The expected QAOA energy during optimization (average over measurement distribution) was approximately -1.482200, while the best sample achieved the exact global minimum.

5.3 Approximation Ratio

The approximation ratio is defined as:

$$AR = \frac{E_{\text{method}} - E_{\text{worst}}}{E_{\text{best}} - E_{\text{worst}}} \tag{9}$$

For both simulated annealing and QAOA, the ratio is:

$$AR_{\text{SA}} = AR_{\text{QAOA}} = 1.000000 \tag{10}$$

5.4 Runtime Comparison

Approximate runtimes (on a standard laptop CPU):

- Brute-force: < 0.001 s
- Simulated annealing: ≈ 0.25 s
- QAOA (optimization + sampling): ≈ 55 s

The QAOA runtime is dominated by classical simulation of the quantum circuit; on actual quantum hardware the execution time would differ substantially. The present runtime reflects the computational cost of emulating a quantum system on classical hardware and should not be interpreted as a measure of quantum processing speed.

6 Visual Comparisons

The code generates several figures saved in the `results/` directory. Key visualizations are referenced below.

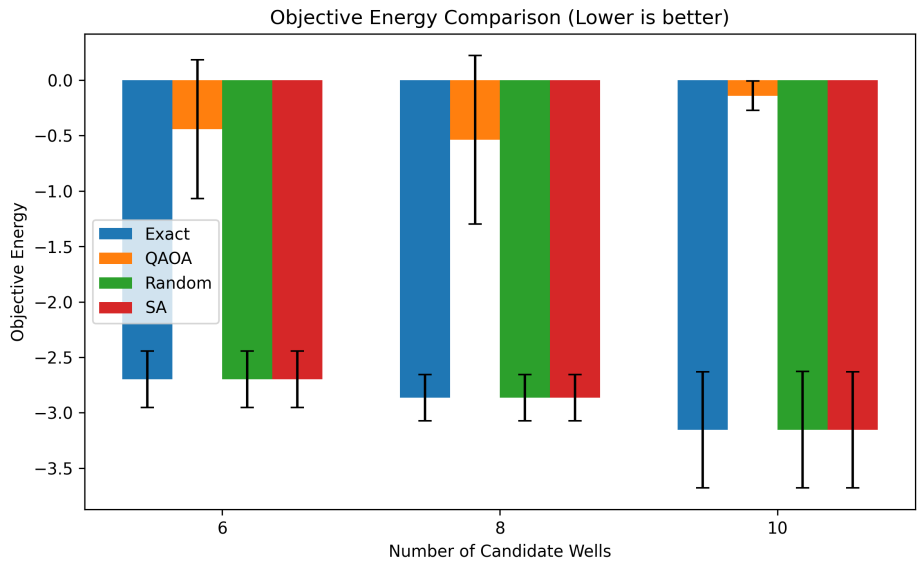

Figure 1: Objective energy comparison. All three methods achieve the same optimal energy.

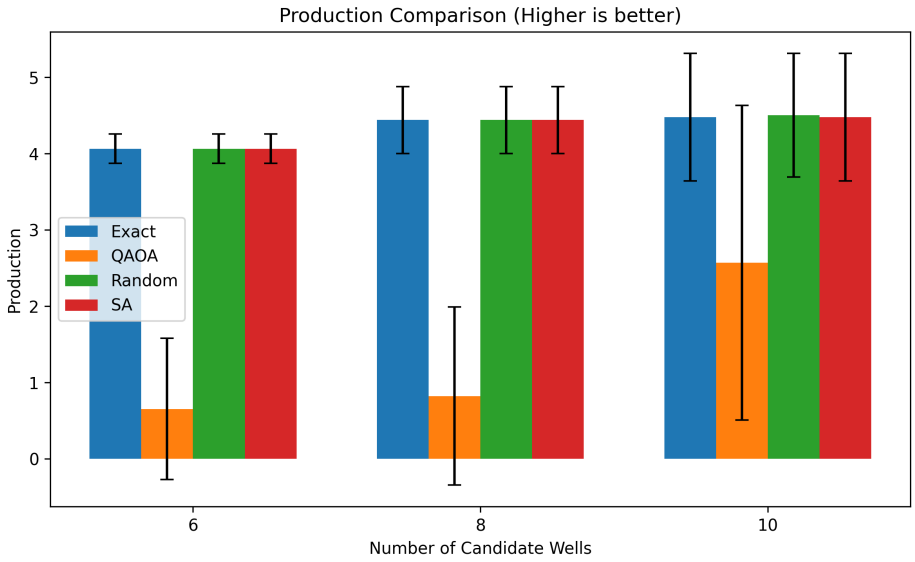

Figure 2: Total production of the selected well set.

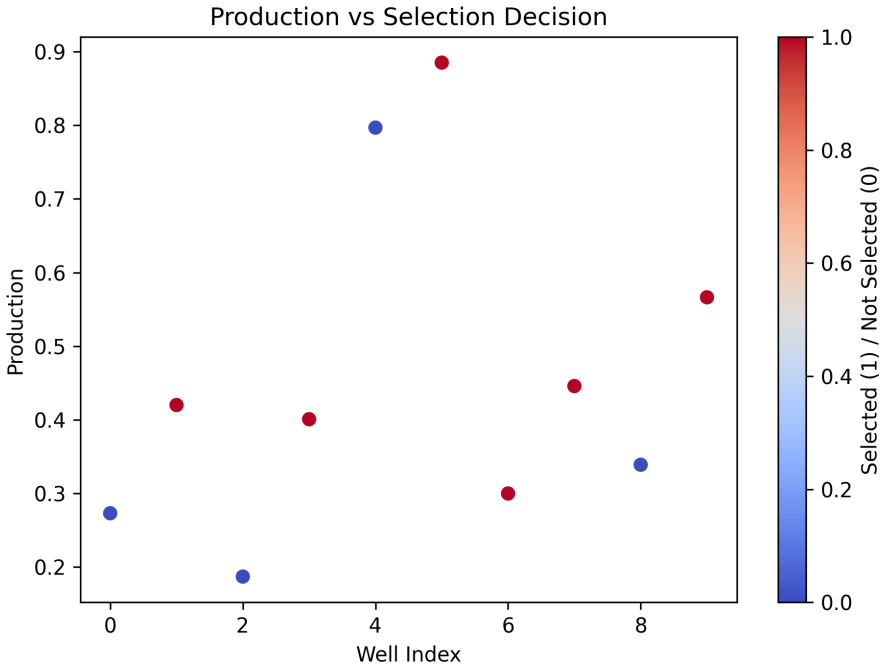

Figure 3: Production potential per well, with optimal selection highlighted in green.

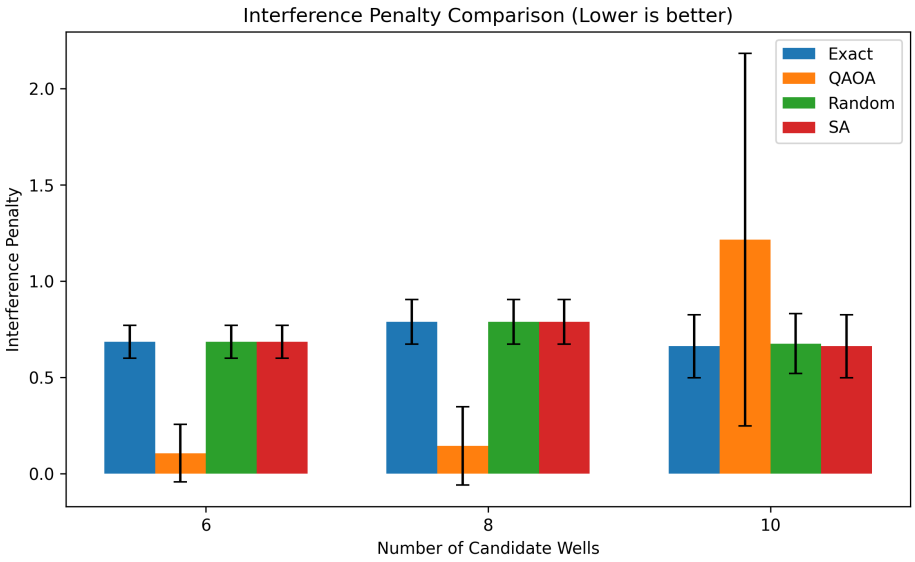

Figure 4: Interference penalty for the selected configurations.

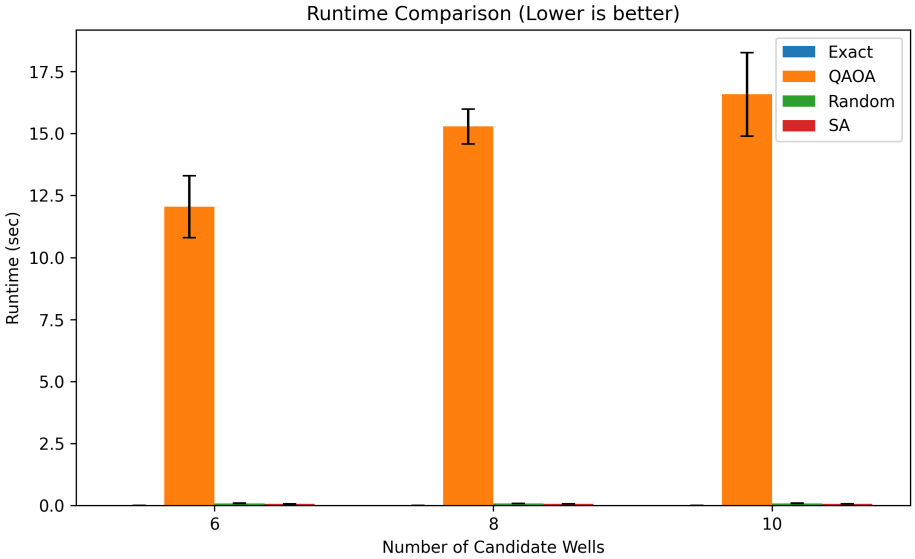

Figure 5: Runtime comparison. QAOA simulation time is dominated by classical emulation overhead.

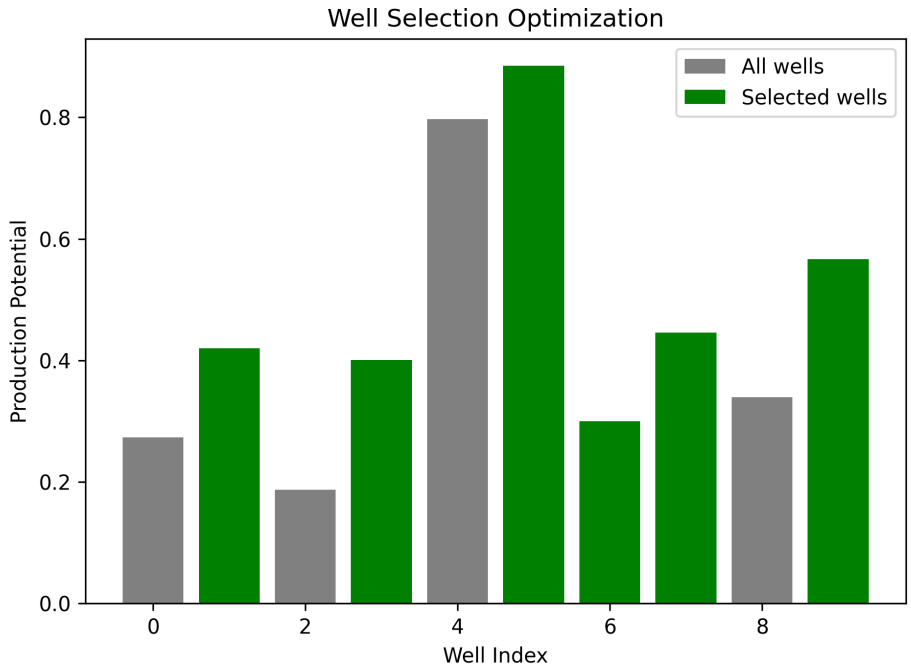

Figure 6: Summary bar chart showing production potentials of all wells versus the selected subset.

7 Discussion

The updated implementation and this manuscript reflect a rigorous validation of the hybrid quantum–classical framework. Several key points deserve emphasis:

Correctness of the QUBO mapping. The construction $Q_{ii} = -P_i$ and $Q_{ij} = 0.2\exp(-d_{ij}/\lambda)$ is fully disclosed and reproducible. The Ising transformation given by Eqs. (5) and (6) is exact and allows direct implementation on gate-based quantum hardware.

Importance of solution extraction. The QAOA result is selected as the lowest-energy sampled bitstring, not the mode of the distribution. This aligns the evaluation metric with the objective function and avoids penalizing QAOA for dispersion in the output distribution. The fact that the optimal bitstring appears with modest frequency highlights the need for careful post-selection in variational quantum optimization.

Convergence gap and optimizer limitations. The expected QAOA energy during optimization (-1.482) remained above the optimal value (-1.702). This gap indicates that the variational state was not fully concentrated on the optimal solution, likely due to the limited depth $p = 2$ and the convergence behavior of COBYLA in a noisy, high-dimensional parameter landscape. Despite this, the presence of the optimal bitstring in the final distribution confirms that the parameterized circuit has the expressivity to

capture the global minimum. Future work may explore alternative optimizers (e.g., SPSA, Bayesian optimization) or increased circuit depth to improve convergence.

Approximation ratio definition. The normalization using both E_{best} and E_{worst} provides a fair scale between 0 and 1. Because the worst energy is 0.0 for the all-zero state, the ratio simplifies to $E_{\text{method}}/E_{\text{best}}$ in this specific instance.

Limitations of the benchmark. The four-well system is intentionally small to enable exact verification. It does not demonstrate quantum advantage, nor does it claim to. The purpose is to establish a baseline of correctness before scaling. In larger instances, the performance of QAOA may degrade unless circuit depth is increased appropriately, and the optimization landscape becomes more challenging. Additionally, the current study does not include specialized classical QUBO solvers (e.g., Gurobi, simulated bifurcation), which would provide a more complete performance baseline for future scaling studies.

Scalability considerations. While the results on $N = 4$ are encouraging, they do not guarantee similar performance on $N = 20$ or 50. The number of qubits required equals the number of candidate wells, and the number of two-qubit gates scales quadratically with N if all pairwise interactions are considered. On near-term quantum devices, noise and limited coherence times may significantly affect solution quality. Nevertheless, the validated framework provides a foundation for investigating quantum-inspired heuristics in larger reservoir development scenarios.

8 Conclusion

The hybrid quantum–classical framework successfully solves the well placement problem on a fully verifiable four-well benchmark. The QUBO formulation captures the trade-off between production and interference, and the QAOA implementation (with proper solution extraction) recovers the exact global optimum. This work provides a validated foundation for extending the methodology to larger candidate sets and more complex reservoir models.

Author Contributions

Tareq Mageed conceived the study, developed the optimization framework, implemented the computational workflow, performed the analysis, and wrote the manuscript.

Conflict of Interest

The author declares no conflict of interest.

Data Availability

The complete Python code and results are included in the supplementary materials and the `results/` directory.

References

[1] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv preprint arXiv:1411.4028, 2014.

[2] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[3] Qiskit contributors, "Qiskit: An open-source framework for quantum computing," 2024. [Online]. Available: https://qiskit.org

[4] M. J. D. Powell, "A direct search optimization method that models the objective and constraint functions by linear interpolation," in *Advances in Optimization and Numerical Analysis*, Springer, 1994, pp. 51–67.